Pre-reading

Five days of Python before the track starts, then one short reading before each class. Ten to fifteen minutes each.

TRACK	Building the Right AI System, 15 classes
COMPANY IN THE EXAMPLES	Lafayette O'Reilly Logistics (invented)
CODE AND DATA	The course repository: `code/`, `data/`

Contents

1 Before the track: Python in five days 3
2 Before class 2: writing a problem down 5
3 Before class 3: train and test 6
4 Before class 4: regular expressions 7
5 Before class 5: attention 8
6 Before class 6: tokens 9
7 Before class 7: JSON and schemas 10
8 Before class 8: our AI policy 11
9 Before class 9: embeddings 12
10 Before class 10: function calling 13
11 Before class 11: MCP 14
12 Before class 12: agents 15
13 Before class 13: evaluation 16
14 Before class 14: running services 17
15 Before class 15: pick your problem 18

© 2026 Devansh Parapalli

1 Before the track: Python in five days

Two topics a day, about 45 minutes each. If you write Python daily, skip to day 5 and run the environment check.

Everything runs with `uv`, which installs Python and each script's packages for you. Install it once, then any script in the repo runs with `uv run path/to/script.py`.

1.1 Day 1: running code, and values

- **Running a script.** Install `uv`, then run `uv run code/class02/place.py`. You should see seven problems placed on the card.
- **Values and variables.** Numbers, text (strings) and true/false. `queue = "invoice-import"` gives a name to a value. Print things to see them: `print(queue)`.

Try: write a script that stores a ticket ID and a queue in two variables and prints one line with both.

1.2 Day 2: lists, dictionaries and loops

- **Lists** hold things in order: `queues = ["payroll", "catering", "hardware"]`.
- **Dictionaries** hold things by name: `ticket = {"id": "TKT-418004", "queue": "shipment-ops"}`.
- **Loops** do something to each item: `for q in queues: print(q)`.

Try: make a list of three ticket dictionaries and print only the ones in `shipment-ops`.

1.3 Day 3: functions, and reading files

- **Functions** give a name to a few lines so you can reuse them: `def is_urgent(ticket): return ticket["priority"] in ("P1", "P2")`.
- **Reading a CSV.** `data/tickets_handwritten.csv` has 300 tickets. Python's `csv` module reads it row by row.

Try: count how many handwritten tickets are P1 or P2.

1.4 Day 4: tables with pandas

- **pandas** reads a CSV into a table in one line: `pd.read_csv("data/tickets_handwritten.csv")`.
- **Counting and filtering.** `df.queue.value_counts()` counts tickets per queue. `df[df.department == "Warehouse"]` keeps only warehouse tickets.

Try: which department raises the most P1 and P2 tickets?

1.5 Day 5: calling a web service, and the environment check

- **HTTP and JSON.** A model provider is a web service. You send JSON, you get JSON back. The class code wraps this in one function, `ask()`.

© 2026 Devansh Parapalli

- **Environment variables** keep keys out of code. Set `LLM_API_KEY`, `LLM_BASE_URL` and `LLM_MODEL` in your shell, never in a file you commit.

Environment check, all three should work before class 1:

1. `uv --version` prints a version.
2. `uv run data/generate.py` writes `data/tickets.csv`.
3. `uv run code/class01/first_call.py` prints an answer from the model.

© 2026 Devansh Parapalli

2 Before class 2: writing a problem down

Most AI projects that go nowhere started with a tool and went looking for a problem. Class 2 turns that around. Before it, practise the first step: writing one problem from your own work so it can be judged.

A good problem statement is one sentence with a verb, and it says what goes in and what should come out:

- **Weak:** "Use AI for customs."
- **Better:** "When customs rejects a shipment's documents, tell the ops coordinator which document is wrong and what's missing."

Then answer, roughly, for your problem:

1. Is the rule already written down anywhere?
2. What goes in: a table, text, or images?
3. Do we have past examples with the right answer attached?
4. What does one wrong answer cost, and who notices?
5. Does the answer have to be exact or explainable?
6. How often does it happen?

Bring the sentence and your six answers to class 2. Don't worry about the answer being "no AI needed". That's often the right one.

© 2026 Devansh Parapalli

3 Before class 3: train and test

A model learns from examples. The obvious way to check it is to ask it about examples it has seen, and that's exactly the mistake.

Think of an exam. If students see the paper beforehand, their marks tell you nothing. So we split our examples: most for learning (training), some kept back and never shown (testing). Only the test score counts.

Two traps to know before class:

- **Leakage.** The answer sneaks into the training data. A ticket that says "moved to the payroll queue" in its text teaches the model to read the answer, not the problem.
- **Accuracy on uneven data.** If 99 in 100 tickets aren't P1, a model that always says "not P1" is 99% accurate and useless.

Question to bring: in your team's work, what would "always guess the most common answer" get right, and how often?

© 2026 Devansh Parapalli

4 Before class 4: regular expressions

A regular expression (regex) describes the shape of a piece of text. Anything at LOL with a fixed shape can be found exactly, every time, for free:

- ticket IDs: `TKT-` then six digits, `TKT-\d{6}`
- shipments: `LOL-SH-` then seven digits, `LOL-SH-\d{7}`
- InvoiceHub batches: `IH-` then six digits, a dash, two digits

Read the patterns in `code/class04/extract.py`. You don't need to write regex fluently; you need to recognise when a problem is "find the thing with a fixed shape" and not reach for a model.

Try: open `data/tickets_handwritten.csv` and find three tickets where a regex would pull out the key detail, and three where it couldn't.

© 2026 Devansh Parapalli

5 Before class 5: attention

Older language models read a sentence one word at a time and forgot the beginning by the end. The idea that changed that is **attention**: every word looks at every other word and decides which ones matter to it.

Take: "The close job failed after the journal import because it timed out." To understand "it", a model with attention weighs "close job" heavily and "the" hardly at all. It does this for every word at once, in many layers.

That's the core of the transformer, the design behind every large language model today. Two consequences we'll use:

- It trains in parallel, which is why models got so big.
- Work grows with the square of the text length, which is why models have a limit on how much they can read at once (class 6).

For the curious: the original paper is "Attention Is All You Need" (Vaswani et al., 2017). Not needed for class.

© 2026 Devansh Parapalli

6 Before class 6: tokens

Models don't read words; they read **tokens**, pieces of text. Common English words are one token. Company names, IDs and other languages split into several.

Tokens matter for three reasons:

- **Cost.** Providers charge per token, in and out.
- **Limits.** The context window, everything the model can see in one request, is counted in tokens.
- **Odd behaviour.** A shipment ID split into eight pieces is one reason models are bad at copying long numbers exactly.

Try: run `uv run code/class06/tokens.py` and see how "Bayerische Unfug Maschinen" splits.

© 2026 Devansh Parapalli

7 Before class 7: JSON and schemas

When a person reads a model’s answer, “close enough” is fine. When a program reads it, it isn’t. So we ask for JSON: named fields with values.

```
{ "queue": "shipment-ops", "shipment_id": "LOL-SH-4472190", "blocking": true }
```

A **schema** says which fields must be there, what type each is, and which values are allowed. The class code uses Pydantic to write the schema as a Python class and check every reply against it. If the reply doesn’t fit, we send the error back and ask again.

Question to bring: pick a task from your team where the output feeds another system. What fields would the schema need?

© 2026 Devansh Parapalli

8 Before class 8: our AI policy

This is the one-page version of LOL's policy. Read it; class 8 is about using it.

Data at LOL is in one of five classes: LOL Public, LOL Internal, LOL Confidential, LOL Highly Confidential, LOL Restricted.

Every approved AI tool has a ceiling, the highest class it may receive:

- **Lighthouse, our internal AI gateway:** up to LOL Restricted
- **ServiceDesk AI, built into the ticketing tool:** up to LOL Highly Confidential
- **Open models running on your own laptop:** up to LOL Confidential
- **Anything else:** LOL Public only

Two of the companies we work with have their own rules, and their data brings those rules with it:

- **Bayerische Unfug Maschinen:** No written AI policy. Microsoft 365 Copilot is switched on in Office, and that's the extent of it. Treat anything BUM sends us as LOL Confidential, whatever they did with it before.
- **Armadillo Security Services:** Strict. Background check data doesn't leave approved systems, ever. Background check results never go into any AI tool.

Question to bring: find one thing you did last week that would have needed a class decided before it went into a tool.

© 2026 Devansh Parapalli

9 Before class 9: embeddings

An **embedding** turns a piece of text into a list of numbers, so that texts with similar meaning get similar numbers. "Invoices not showing" and "import stuck" end up close together even with no words in common.

Class 5 used embeddings to classify tickets. Class 9 uses them to search: turn the question into numbers, find the runbook passages with the closest numbers, and hand those to the model to answer from.

Keyword search still matters. It's better at exact things like `IH-260615-11` or an error code. The best search uses both.

Try: read two runbooks in `data/runbooks/` and write one question each could answer, worded differently from the runbook itself.

© 2026 Devansh Parapalli

10 Before class 10: function calling

A model on its own can only produce text. **Function calling** lets it ask our code to do something: look up a batch in InvoiceHub, find a shipment in Tracklane, open a ticket.

The important part: the model never touches the system. It asks, in a fixed format, for a function by name with some arguments. Our code decides whether to run it, runs it, and hands back the result.

That split is where safety lives. Reading is usually fine. Anything that changes money, people’s access or what leaves the building waits for a person.

Question to bring: list three lookups your team does by hand every day that a model could ask for, and one action it should never take on its own.

© 2026 Devansh Parapalli

11 Before class 11: MCP

Every AI app needs to reach our systems, and every system would need its own connector for every app. The Model Context Protocol (MCP) is a standard plug: each system exposes one MCP server, and any app that speaks MCP can use it.

A server can offer three things: **tools** (actions to ask for), **resources** (things to read, like a runbook) and **prompts** (ready-made instructions).

One thing to keep in mind before class: an MCP server is software you install and run. It can do whatever it's allowed to. That's why servers go through the same approval as any other tool.

The specification and examples are at modelcontextprotocol.io.

© 2026 Devansh Parapalli

12 Before class 12: agents

"Agent" gets used for almost anything. In this track it means four things together: a goal, a loop (look, decide, act, look again), tools, and a reason to stop.

Class 12 runs one on a real-looking case: a customs hold at Nhava Sheva. A supervisor agent decides it's a customs problem and hands it to a worker agent, which looks up the shipment, finds the runbook step and drafts an email. A person reads the draft and decides whether it goes.

Question to bring: for one multi-step task in your team, where would you put the person? Before what step does a mistake become expensive?

© 2026 Devansh Parapalli

13 Before class 13: evaluation

A demo shows the cases that work. An evaluation shows how often it's wrong.

The method is plain:

1. Collect real examples, including the messy ones.
2. Write down the right answer for each, ideally with two people agreeing.
3. Freeze the set. Score every change against the same set.
4. Add every mistake you find later.

We already have the start of one: the 300 handwritten tickets. In class 3, a simple classifier got 71% of them right. Anything more expensive has to beat that, or be better at something we can name.

Question to bring: for an AI feature your team wants, who would own it when it's wrong? Put a name, not a team.

© 2026 Devansh Parapalli

14 Before class 14: running services

A script that worked once on a laptop isn’t a service. Before class, think about what you’d need to know at 2 am if it broke:

- Which version of the prompt and model was running?
- What did it cost yesterday, and is today different?
- How slow is it, and is that getting worse?
- How do you switch it off, or back to the last version, in one step?

Class 14 sets up a container, a trace viewer and a cost log that answer these. It also covers change windows. Finance systems at LOL don’t change during period close.

© 2026 Devansh Parapalli

15 Before class 15: pick your problem

Class 15 ends with the capstone. Teams place a problem nobody has seen on the card and write a one-page proposal for the cheapest thing that works.

To prepare:

1. Reread your class 2 problem statement. Would you place it the same way now?
2. Look at the example proposal and the blank one. The blank is the one page you'll fill in, however you like: the problem, where it lands, steps, error budget, cost, owner and off switch.
3. If you already have a better problem than anything in the draw, bring it. A team can use its own.
4. Remember "no AI needed" is a valid answer, and sometimes the winning one.

© 2026 Devansh Parapalli